

# **PROJECT REPORT**

Module 6: In-Vehicle Infotainment (IVI) Systems

## **Conceptual Architecture Design of an In-Vehicle Infotainment (IVI) System Integrating Media, Navigation and Projection Features**

*with Annotated Component Interfaces, Data Flow and a Browser-Based Signal-Flow  
Simulation*

**SUBMITTED BY**

**Lopamudra Roy**

Enrollment No.: 12023002001195

**Department of Computer Science and Engineering**

Institute of Engineering and Management, Kolkata

Academic Session: 2023–2027

## **Contents**

1. Title Page
2. Abstract
3. Introduction
4. Background / Literature
5. Tools & Environment
6. System Design / Methodology
7. Implementation / Modelling
8. Testing & Results
9. Discussion
10. Conclusion & Future Scope
11. References
12. Appendix

## 2. Abstract

An In-Vehicle Infotainment (IVI) system is the software and hardware platform that brings entertainment, navigation, connectivity and vehicle information to the driver through a single interface. Modern IVI platforms must combine several independent features without letting them interfere with each other, and they must remain safe and predictable when events such as an incoming call interrupt what the driver is doing.

This project addresses the task: "Design a conceptual architecture diagram of an IVI system integrating media, navigation, and projection features. Annotate component interfaces and data flow." A layered architecture is proposed in which a driver and a paired smartphone interact with an IVI HMI, an IVI Controller routes every request, and dedicated Media, Navigation and Projection services deliver the features. Hardware Abstraction Layers (HALs) connect the services to speakers and the display or HUD, while a CAN/LIN vehicle network links the controller to the vehicle ECUs.

The architecture diagram is annotated with the interface and the data carried on every connection. To validate the design, a dependency-free, browser-based simulation called "Cabin Relay" was built with HTML, CSS, JavaScript and generated SVG. It animates five scenarios (play music, start navigation, project route, incoming call, end call) and enforces the system rules. All scenarios behaved as specified. The work is conceptual and educational; no real CAN bus, GPS, HAL or ECU is used.

**Keywords:** In-Vehicle Infotainment, IVI, system architecture, Media Service, Navigation Service, Projection Service, HAL, CAN, Android Auto, CarPlay, simulation.

## 3. Introduction

### 3.1 What an IVI system is and why it matters

An In-Vehicle Infotainment (IVI) system is the hardware and software platform in a car that delivers entertainment, navigation, communication and vehicle information through one touch, voice and display interface. The dashboard of a modern car is no longer only a set of gauges; it hosts a touch display, voice control, streaming audio, live maps and smartphone integration.

**Driver experience:** a single, consistent interface for music, maps and calls makes the car easier to use, and projection of Android Auto or CarPlay brings the familiar phone experience to the vehicle.

**Safety:** the driver is also operating a vehicle, so responses must be quick, the interface must keep distraction low, and events such as an incoming call must be handled predictably, for example by pausing music rather than letting audio overlap.

**Connectivity:** the IVI system links the driver, the smartphone, satellite positioning and the vehicle network (CAN/LIN), so information from all of them can be shown in one place.

### 3.2 Objective of the chosen option

Out of the four options in Module 6, Option 1 was chosen: "Design a conceptual architecture diagram of an IVI system integrating media, navigation, and projection features. Annotate component interfaces and data flow." The objectives are:

- To identify the main components of an IVI system and group them into clear layers.
- To define the interface and the data exchanged between every pair of connected components.
- To describe the end-to-end data flow for media, navigation, projection, call handling and vehicle data.
- To validate the architecture with an interactive simulation that follows the same components and rules.

### 3.3 Scope and limitations

The project is a conceptual design and a simulation only. It shows how components are organised and how information moves between them. It does not implement a production IVI stack: no real CAN/LIN vehicle bus, GPS receiver, Android Automotive, HAL or ECU is used, and all timestamps are simulated.

## **4. Background / Literature**

### **4.1 What an IVI system does**

An IVI system combines infotainment functions (music, radio, streaming), driver assistance information (navigation and maps), communication (calls and messages) and vehicle status in a single head unit. It usually runs on an embedded SoC with a real-time or Linux-based operating system, or on Android Automotive OS.

#### **4.1.1 Layered software architecture**

IVI software is commonly organised in layers: applications and HMI at the top, middleware and services in the middle, and hardware abstraction and drivers at the bottom. This separation lets a service such as media playback be developed and replaced without changing the display or the audio hardware underneath. The Hardware Abstraction Layer (HAL) is the boundary that hides device-specific details from the services above it. Industry consortia such as COVESA (formerly GENIVI) and standards such as AUTOSAR promote this kind of modular design.

### **4.2 Smartphone projection**

Android Auto and Apple CarPlay let a paired phone run compatible apps and show their interface on the car display. The phone performs the computation and the head unit acts as the screen and input device, which keeps the car experience up to date as phones change.

### **4.3 Vehicle networks**

Vehicle electronic control units (ECUs) exchange data over in-vehicle networks. The Controller Area Network (CAN) is a robust message-based bus widely used for signals such as speed and gear. The Local Interconnect Network (LIN) is a lower-cost bus used for simple sensors and actuators. An IVI system normally reaches these networks only through a controlled gateway, because unrestricted access from an entertainment unit would be a safety and security risk.

### **4.4 Navigation**

Navigation combines two independent sources: a live position from a GNSS/GPS receiver and stored map data with points of interest. The navigation service merges them to compute a route, and the route can then be shown on the main display, an instrument cluster or a head-up display (HUD).

### **4.5 Summary of the gap addressed**

Descriptions of IVI systems are often either very high-level marketing diagrams or very detailed vendor documents. This project sits between the two: it presents a clear conceptual architecture with each interface and data flow annotated, and backs it with a working simulation so that the behaviour can be observed rather than only read.

## 5. Tools & Environment

**Table 1: Tools and technologies used**

| Item                      | Purpose                                                                                                           |
|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| Architecture diagram tool | Layered block diagram of the IVI architecture with annotated interfaces and flow (Figure 1).                      |
| HTML5                     | Page structure of the "Cabin Relay" simulation (index.html).                                                      |
| CSS3                      | "Engineer's notebook" design system: colour tokens, components, SVG states and responsive layout (css/theme.css). |
| JavaScript (ES2017+)      | Store, scenario engine, journal and animation logic in six small modules.                                         |
| SVG (generated)           | The signal map is generated at run time from a topology data file.                                                |
| Web browser               | Any modern browser; the simulation runs offline by opening index.html, with no server or internet.                |

The simulation has no external libraries or network dependencies, so it can be demonstrated on any computer.

## 6. System Design / Methodology

### 6.1 Design approach

The design follows a top-down method. First the user-facing entry points were identified (driver and smartphone). Next a single orchestration point, the IVI Controller, was placed between the HMI and the feature services so that all requests are routed and checked in one place. Then each feature was given its own service, and each service was connected to the hardware it needs through a HAL or a data source. Finally the vehicle network was added as a separate path with its own labelled flow.

### 6.2 Conceptual architecture

Figure 1 shows the proposed IVI system architecture. Solid arrows are data and control flow. The dashed arrow between the Navigation Service and the Projection Service marks an external data flow.

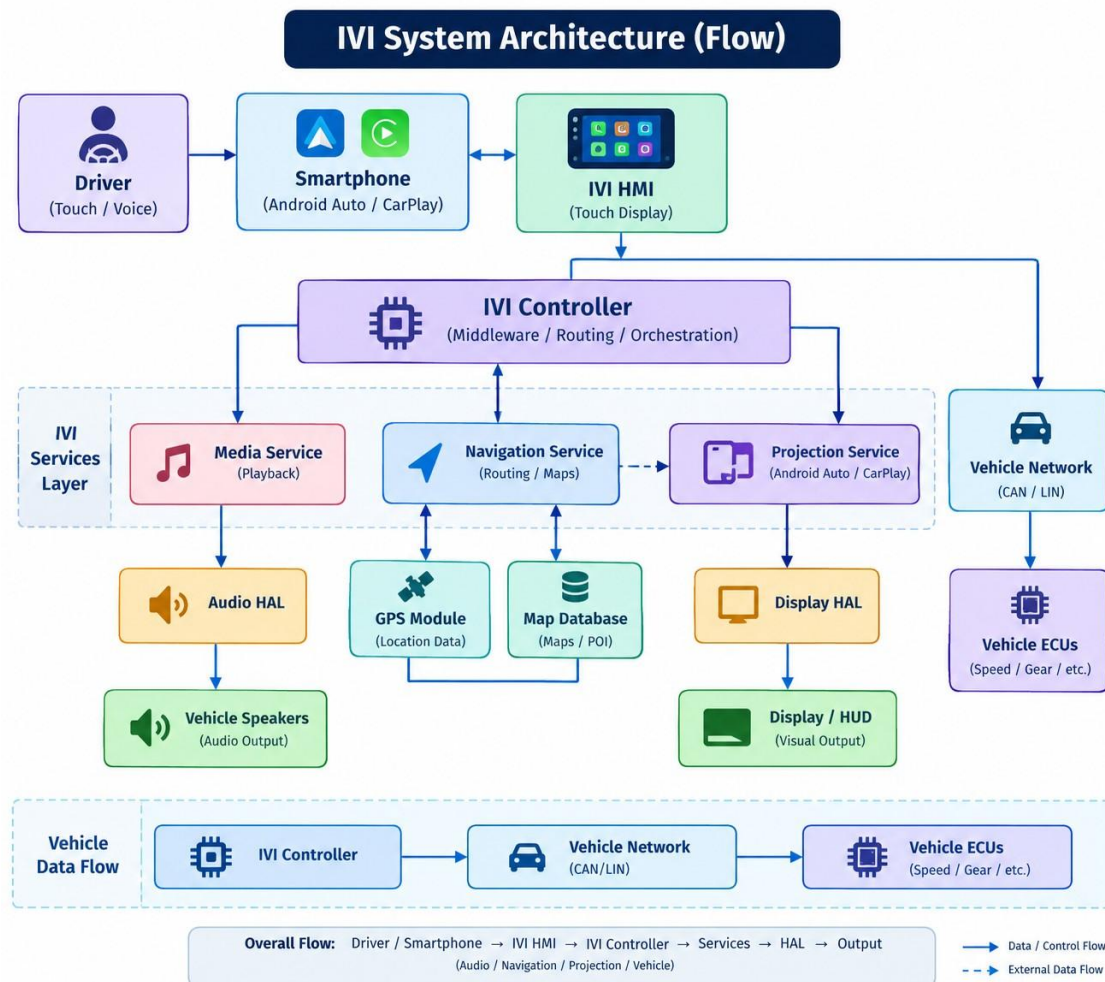


Figure 1: IVI system architecture (flow) integrating media, navigation and projection

## 6.3 Layers of the architecture

**Table 2: Layers and their components**

| Layer                | Components                                  | Responsibility                                              |
|----------------------|---------------------------------------------|-------------------------------------------------------------|
| User and external    | Driver, Smartphone (Android Auto / CarPlay) | Source of touch, voice and projected content.               |
| Presentation         | IVI HMI (touch display)                     | Shows the interface and captures commands.                  |
| Middleware           | IVI Controller                              | Routing, orchestration and rule checking for all requests.  |
| Services             | Media, Navigation, Projection               | Feature logic for playback, routing and projection.         |
| Data sources         | GPS Module, Map Database                    | Live location and stored map/POI data for navigation.       |
| Hardware abstraction | Audio HAL, Display HAL                      | Device-independent interface to audio and display hardware. |
| Output devices       | Vehicle Speakers, Display / HUD             | Audible and visual output to the occupants.                 |
| Vehicle side         | Vehicle Network (CAN / LIN), Vehicle ECUs   | Exchange of vehicle data such as speed and gear.            |

## 6.4 Component descriptions

- **Driver:** interacts by touch or voice; the only human input to the system.
- **Smartphone:** provides Android Auto or CarPlay content and call events, and exchanges data with the HMI in both directions.
- **IVI HMI:** the touch display and user interface. It receives user input and forwards commands to the controller.
- **IVI Controller:** the middleware hub. It routes each command to the right service, enforces system rules (for example, route projection needs active navigation) and forwards vehicle data to the vehicle network.
- **Media Service:** handles playback and produces the audio stream.
- **Navigation Service:** calculates routes by combining GPS position and map data.
- **Projection Service:** prepares smartphone and route content for the display.
- **GPS Module and Map Database:** supply location data and map/POI data respectively.
- **Audio HAL and Display HAL:** convert service output into the form required by the speakers and the display or HUD.
- **Vehicle Network and ECUs:** CAN/LIN transport and the control units that provide values such as speed and gear.

## 6.5 Annotated component interfaces

Each connection in the architecture is an interface with a defined direction and payload. Table 3 lists them; the same labels are used on the simulation's signal map.

**Table 3: Component interfaces and data carried**

| #  | From               | To                 | Direction         | Interface / data                                                |
|----|--------------------|--------------------|-------------------|-----------------------------------------------------------------|
| 1  | Driver             | IVI HMI            | One-way           | Touch / voice input                                             |
| 2  | Smartphone         | IVI HMI            | Two-way           | Media, call events and projection data (Android Auto / CarPlay) |
| 3  | IVI HMI            | IVI Controller     | One-way           | User commands                                                   |
| 4  | IVI Controller     | Media Service      | One-way           | Media command (play, pause, resume)                             |
| 5  | IVI Controller     | Navigation Service | Two-way           | Navigation request and status                                   |
| 6  | IVI Controller     | Projection Service | One-way           | Projection command                                              |
| 7  | Media Service      | Audio HAL          | One-way           | PCM audio stream                                                |
| 8  | Audio HAL          | Vehicle Speakers   | One-way           | Audio output signal                                             |
| 9  | GPS Module         | Navigation Service | Two-way           | Location data                                                   |
| 10 | Map Database       | Navigation Service | Two-way           | Map tiles and POI data                                          |
| 11 | Navigation Service | Projection Service | Dashed / external | Route data                                                      |
| 12 | Projection Service | Display HAL        | One-way           | Display content                                                 |
| 13 | Display HAL        | Display / HUD      | One-way           | Video output                                                    |
| 14 | IVI Controller     | Vehicle Network    | One-way           | Vehicle data over CAN                                           |
| 15 | Vehicle Network    | Vehicle ECUs       | One-way           | CAN / LIN frames (speed, gear)                                  |

## 6.6 Data flow

The overall flow of the system is: Driver or Smartphone → IVI HMI → IVI Controller → Services → HAL → Output. The individual flows are described below.

**Table 4: End-to-end data flows**

| Flow             | Path                                                                         |
|------------------|------------------------------------------------------------------------------|
| Media playback   | Driver → HMI → Controller → Media Service → Audio HAL → Vehicle Speakers     |
| Navigation       | Driver → HMI → Controller → Navigation Service ← GPS Module and Map Database |
| Route projection | Navigation Service → Projection Service → Display HAL → Display / HUD        |
| Incoming call    | Smartphone → HMI → Controller → Media Service (pauses playback if playing)   |
| Vehicle data     | IVI Controller → Vehicle Network (CAN/LIN) → Vehicle ECUs                    |

## 6.7 System rules

The following behavioural rules are part of the design and are enforced in the simulation:

1. Project Route requires active navigation; otherwise the request is rejected.
2. An incoming call pauses playing media.
3. End Call restores music only if the call paused it.

4. Play Music is rejected while a call is in progress, so that the state can never become inconsistent.
5. Controls are locked while a scenario is running, so runs never overlap.
6. Reset returns the system to the idle state.

## 7. Implementation / Modelling

### 7.1 Overview of "Cabin Relay"

To validate the architecture, an interactive simulation named Cabin Relay was built. It shows the same components on a signal map, animates data packets along each interface and records every step in a timed journal. The user chooses one of five scenarios from a scenario deck; the map, the live state board and the journal update together.

### 7.2 Code organisation

**Table 5: Source files of the simulation**

| File            | Role                                                                                     |
|-----------------|------------------------------------------------------------------------------------------|
| index.html      | Page skeleton: header, scenario deck, signal map, live state, journal and about section. |
| css/theme.css   | Design system: tokens, components, SVG node/link states and responsive layout.           |
| js/topology.js  | Data only: zones, 15 nodes and 15 links from which the diagram is generated.             |
| js/store.js     | Small observable store and a pure function that derives display values from the state.   |
| js/diagram.js   | Builds the SVG, sets node and link states and animates the packets.                      |
| js/journal.js   | Simulated-clock event journal with an empty state.                                       |
| js/scenarios.js | Declarative scenarios: traces, guards and outcomes.                                      |
| js/app.js       | Engine: runs traces, locks controls and renders from state.                              |

### 7.3 Design decisions

- The diagram is generated from data (topology.js). Changing the architecture means editing data, not drawing code.
- Scenarios are declarative traces of the form [link, node, log message]. A single engine plays every scenario, which keeps the behaviour consistent.
- The system state (media, navigation, projection, call, pending, busy) lives in one store, and the interface is rendered from it. This prevents the display from disagreeing with the logic.
- Guards at the start of each scenario enforce the system rules before any animation begins.

### 7.4 System state model

**Table 6: State variables**

| Variable | Values                     | Meaning                              |
|----------|----------------------------|--------------------------------------|
| media    | stopped   playing   paused | Playback state of the Media Service. |

| Variable | Values                   | Meaning                                            |
|----------|--------------------------|----------------------------------------------------|
| nav      | true   false             | Whether navigation is active.                      |
| proj     | true   false             | Whether the route is projected to the display/HUD. |
| call     | none   incoming   active | Phone call state.                                  |
| pending  | text   null              | Transient mode label while a scenario runs.        |
| busy     | scenario id   null       | Locks all controls during a run.                   |

The overall Mode shown on the live state board is derived from these values: Idle, Music, Navigation, Music + navigation, or Call (call has priority).

## 8. Testing & Results

### 8.1 Test method

Each scenario was run in a browser, and both the animation on the signal map and the journal entries were compared with the expected path in Table 4. Rule checks were made by running scenarios in an order that should be rejected. The screenshots below were captured from the running simulation.

### 8.2 Scenario tests

**Table 7: Functional test cases and results**

| #  | Scenario                                | Expected behaviour                                                             | Observed result                                                     | Pass |
|----|-----------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------|------|
| 1  | Play music                              | Path Driver → HMI → Controller → Media → Audio HAL → Speakers; media = Playing | Journal ended with "MUSIC PLAYING"; media gauge Playing             | Yes  |
| 2  | Start navigation                        | GPS and Map Database both feed Navigation; route calculated; nav = Active      | Both sources received; "Route calculated"; nav Active               | Yes  |
| 3  | Project route (nav active)              | Nav → Projection → Display HAL → HUD; proj = Active                            | Journal ended with "ROUTE PROJECTED"                                | Yes  |
| 4  | Project route (nav inactive)            | Rejected with a message to start navigation first                              | Rejected: "PROJECT ROUTE rejected — please start navigation first." | Yes  |
| 5  | Incoming call while music plays         | Media paused; call = Active; mode = Call                                       | "CALL ACTIVE — MUSIC PAUSED"; media gauge Paused                    | Yes  |
| 6  | End call after a call that paused music | Music resumes; call = none                                                     | "MUSIC RESUMED"; media gauge Playing                                | Yes  |
| 7  | End call when music was not playing     | Call ends; media unchanged; nothing resumes                                    | Call ended without starting playback                                | Yes  |
| 8  | End call with no call                   | Rejected                                                                       | Message "No call to end."                                           | Yes  |
| 9  | Play music during a call                | Rejected to keep state consistent                                              | Message "Music can't start during a call. End the call first."      | Yes  |
| 10 | Click during a run                      | Controls locked; no overlapping run                                            | All scenario buttons and Reset disabled while running               | Yes  |
| 11 | Reset                                   | Return to idle; map cleared                                                    | All gauges return to idle values; journal shows "Simulation reset"  | Yes  |

### 8.3 Project Image

Figures 2 to 5 show the simulation after four of the scenarios. In each case the signal map shows completed paths, the state board shows the derived values, and the journal shows the ordered events.

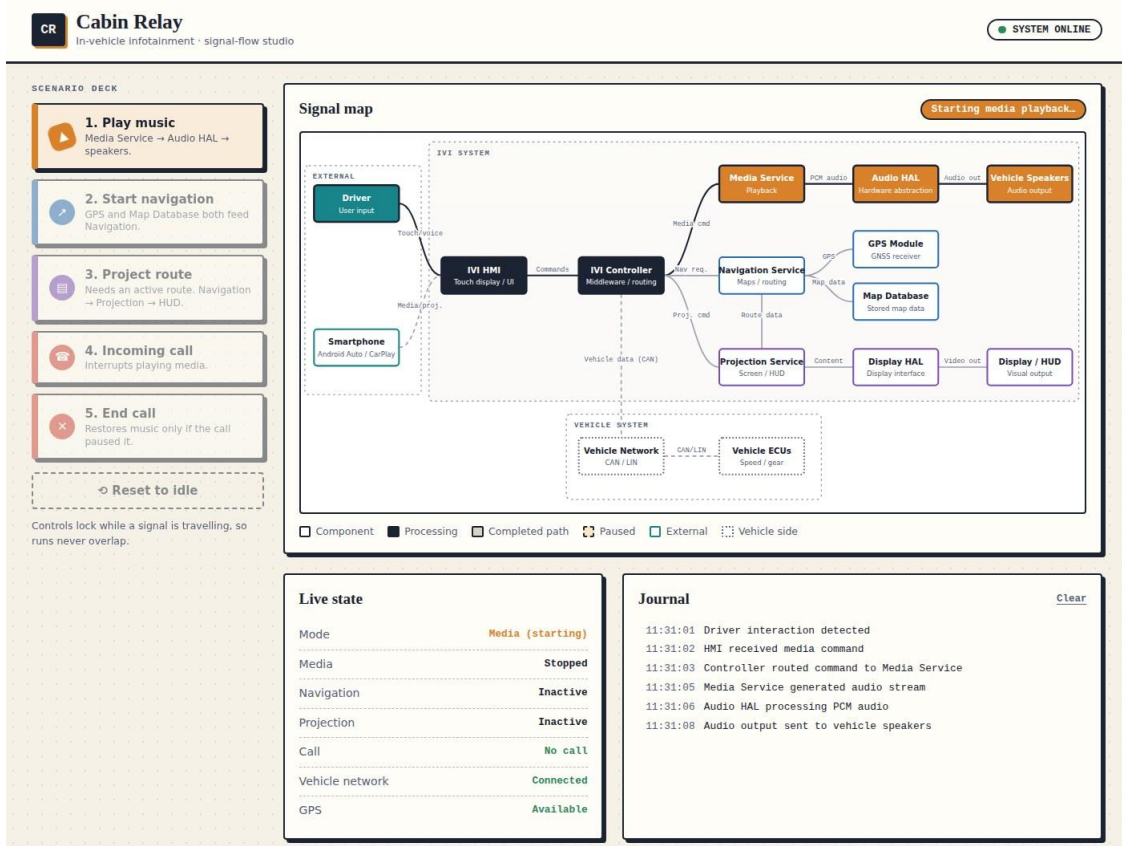


Figure 2: After "Play music" – the path to the speakers is complete and media is Playing

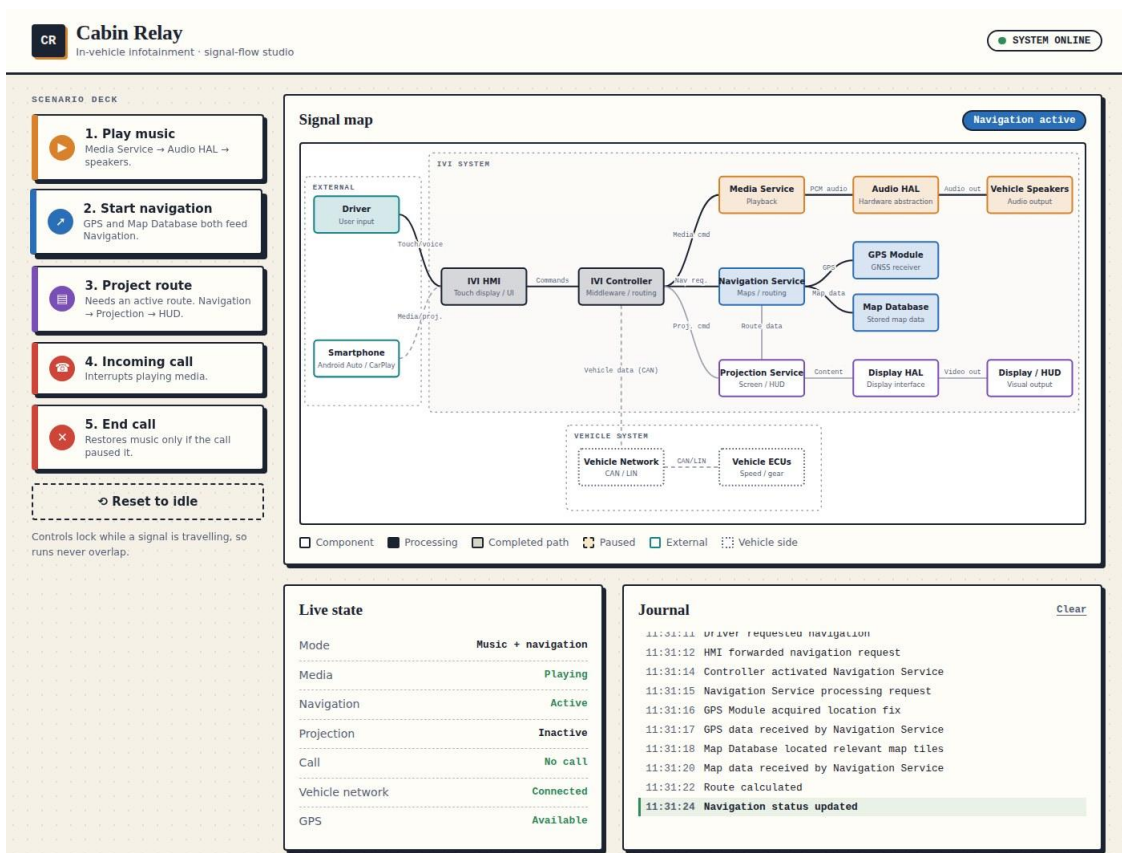


Figure 3: After "Start navigation" – GPS and Map Database both feed the Navigation Service

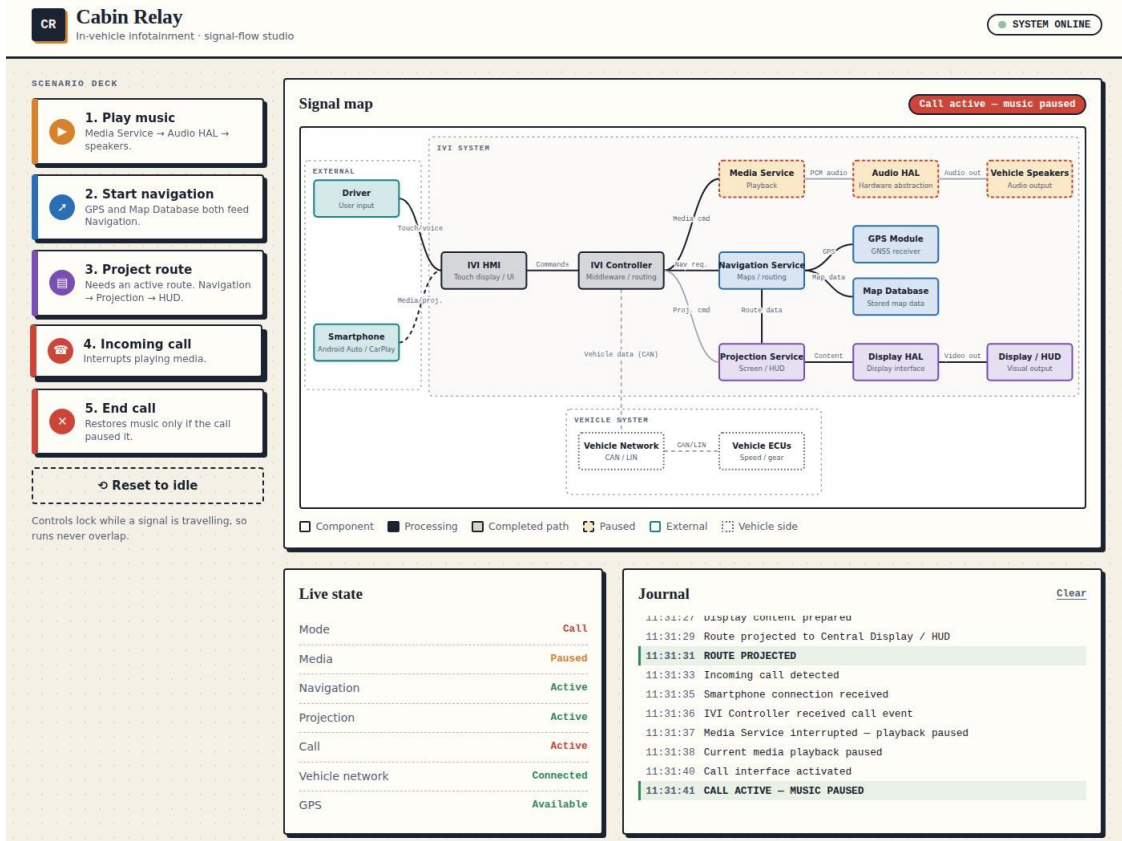


Figure 4: After "Incoming call" – Media Service, Audio HAL and speakers are paused

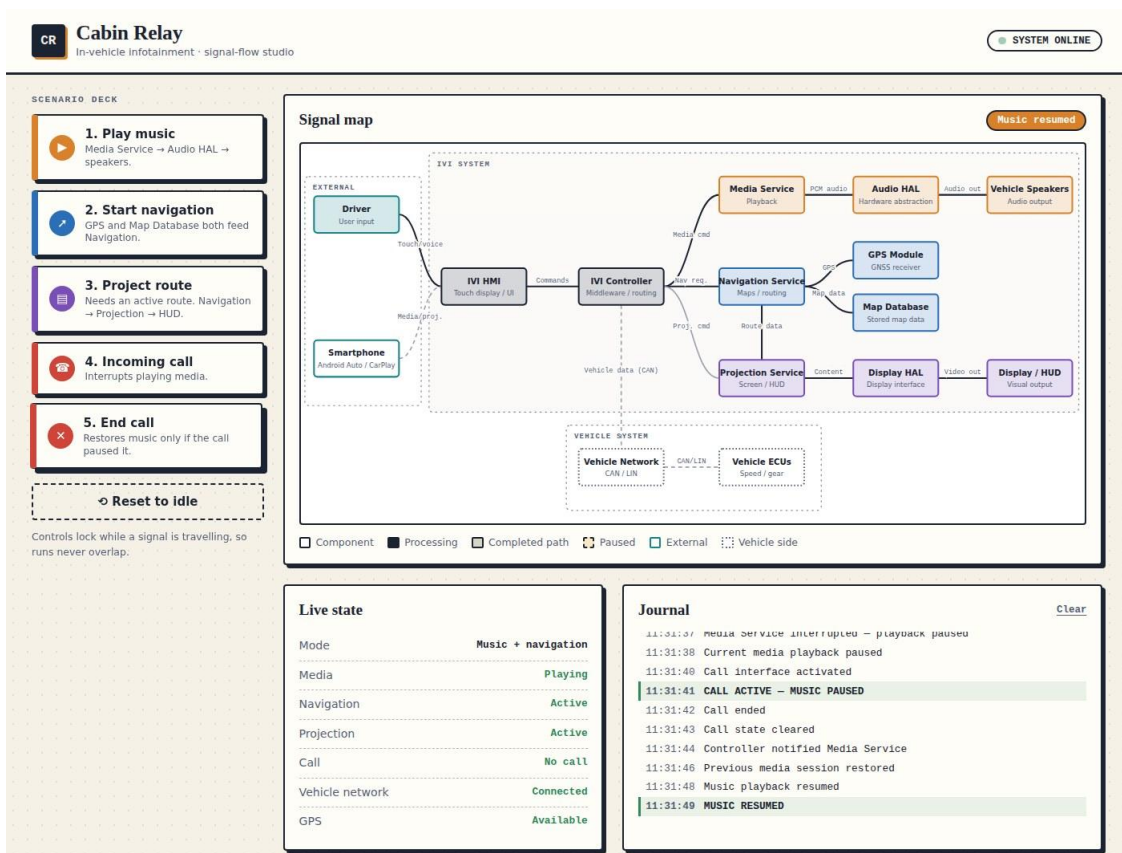


Figure 5: After "End call" – the interrupted music session is restored

## 8.4 Results

All eleven test cases passed. The simulation follows the architecture exactly: every animated hop corresponds to an interface in Table 3, and every scenario follows a path listed in Table 4. The rule checks confirm that the controller-centred design keeps the system in a consistent state even when requests arrive in an unexpected order.

## 9. Discussion

- **Central controller:** routing every request through the IVI Controller gives one place to enforce rules, such as requiring navigation before projection. The trade-off is that the controller becomes a critical component and must be designed for reliability.
- **Layering and HALs:** because services talk to hardware only through a HAL, speakers or displays can change without altering the Media or Projection services.
- **Two-source navigation:** showing GPS and the Map Database as separate inputs makes it clear that route calculation needs both live position and stored data.
- **Call handling:** the resume-only-if-paused rule avoids the common fault of starting music that the driver never had playing. Rejecting Play Music during a call fixed an inconsistent state that a naive design would allow.
- **Vehicle data path:** the vehicle network is drawn as a separate, controlled path from the controller. In a real vehicle this gateway should be read-limited and protected for safety and security.
- **Difference between the diagram and the simulation:** in the architecture diagram the driver interacts with the smartphone as well as the HMI, while the simulation shows the driver reaching the HMI directly and the smartphone connecting to the HMI. This keeps the animation simple without changing the component set.

### 9.1 Limitations

- Conceptual only: no real CAN, GPS, HAL, ECU or Android Automotive is used.
- Timing is simulated, so real latency, buffering and audio focus are not modelled.
- Security, functional safety and error handling (for example GPS loss) are not modelled.

## **10. Conclusion & Future Scope**

### **10.1 Conclusion**

A layered conceptual architecture for an IVI system that integrates media, navigation and projection features was designed. Every component interface and data flow was annotated, and the whole design was checked with a working browser simulation. The results show that a controller-centred, service-based architecture with HAL boundaries can integrate the three feature areas cleanly while handling interruptions, such as an incoming call, in a predictable way.

### **10.2 Future scope**

- Add voice control as a second input path and model its recognition service.
- Model audio focus with priorities for navigation prompts, calls and media.
- Add fault scenarios such as GPS loss or a disconnected phone.
- Extend the vehicle data path with speed-dependent restrictions (for example, locking some inputs while moving).
- Model the security gateway between the IVI system and the vehicle network.
- Move from the conceptual model to a prototype on Android Automotive OS with real HAL interfaces.

## 11. References

- [1] Google, "Android Automotive OS overview," Android Open Source Project documentation, [source.android.com](https://source.android.com).
- [2] Google, "Android Auto for developers," [developer.android.com](https://developer.android.com).
- [3] Apple Inc., "CarPlay," [apple.com/ios/carplay](https://apple.com/ios/carplay).
- [4] AUTOSAR, "AUTomotive Open System ARchitecture – Standards," [autosar.org](https://autosar.org).
- [5] COVESA (formerly GENIVI Alliance), "Vehicle software and infotainment specifications," [covesa.global](https://covesa.global).
- [6] Robert Bosch GmbH, "CAN Specification, Version 2.0," 1991.
- [7] ISO 11898-1, "Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling."
- [8] ISO 17987, "Road vehicles – Local Interconnect Network (LIN)."
- [9] W3C, "Scalable Vector Graphics (SVG) 1.1 Specification," [w3.org/TR/SVG11](https://www.w3.org/TR/SVG11).
- [10] Course material, Module 6: In-Vehicle Infotainment (IVI) Systems.

## 12. Appendix

### Appendix A: Topology data (excerpt from js/topology.js)

```
nodes: driver, phone, hmi, controller, media, nav, proj, audiohal, gps, mapdb,
displayhal, speakers, hud, vehnet, ecus
zones: External, IVI system, Vehicle system
links (from → to : label):
  driver → hmi : Touch/voice           phone → hmi : Media/proj. (dashed)
  hmi → controller : Commands           controller → media : Media cmd
  controller → nav : Nav req.           controller → proj : Proj. cmd
  controller → vehnet : Vehicle data (CAN) (dashed)
  vehnet → ecus : CAN/LIN (dashed)      nav → proj : Route data
  media → audiohal : PCM audio           audiohal → speakers : Audio out
  gps → nav : GPS                       mapdb → nav : Map data
  proj → displayhal : Content            displayhal → hud : Video out
```

### Appendix B: Scenario trace – "Play music" (excerpt from js/scenarios.js)

```
[null, "driver", "Driver interaction detected"]
["driver-hmi", "hmi", "HMI received media command"]
["hmi-controller", "controller", "Controller routed command to Media Service"]
["controller-media", "media", "Media Service generated audio stream"]
["media-audiohal", "audiohal", "Audio HAL processing PCM audio"]
["audiohal-speakers", "speakers", "Audio output sent to vehicle speakers"]
```

### Appendix C: How to run the simulation

1. Extract IVI\_SIMULATION\_Lopamudra.zip.
2. Open the cabin-relay folder and double-click index.html in any modern browser.
3. Click a scenario in the Scenario deck and watch the signal map, live state and journal.  
Use "Reset to idle" to start over.